

Demo guide: IRVE Load Simulator

A live map of Ile-de-France where the charging model's error metrics move as a simulated fleet drives, plugs in and leaves. This is the running order, the lines worth saying out loud, and the questions you will get.

Run it

```
python3 train_for_sim.py
python3 -m sim.roads          # one-off: street routing for vehicles
python3 -m uvicorn sim.server:app --port 8008
```

Open **http://localhost:8008** in a real browser window. Training takes about 20 seconds and only needs re-running if the corpus changes; after that the uvicorn line alone is enough.

START IT 10 MINUTES BEFORE YOU PRESENT

The rolling metrics need 72 simulated hours to fill their window, and at 8 seconds per simulated hour that is about 10 minutes of wall clock. A cold start shows "accumulating hours..." over an empty chart, which is a weak opening. Widen the browser past 900px or the layout stacks and the map gets cramped.

Check it is healthy without leaving the terminal:

```
curl -s localhost:8008/health
```

"status":"running" with a rising `sim_hours` means you are good. "routed":true from `/api/meta` confirms street routing is active. To stop: Ctrl-C, or kill `$(lsof -ti:8008)`.

What is on screen

Element	Meaning
Blue-ringed dots	The 6 sites . This is where the model predicts.
Small grey dots	The 71 individual charge points .
Orange dots	Vehicles driving to charge .
Green dots	Vehicles plugged in .
Blue dots	Vehicles finished, driving away .
Faint orange lines	Active vehicle routes, following real streets.
Site colour	Live absolute error: green, amber, red.
Site halo size	Actual load that hour.

The clock runs at **1 simulated hour per 8 seconds**, so a full day passes in about 3 minutes.

Run the demo in this order

Total about 3 minutes 10 seconds of talking. Everything else is letting it run.

1 Open with the honesty, not the visuals

20 sec

Point at the two badges in the header before anything else.

"Two labels up front. Amber says the model was trained on synthetic sessions - the real corpus isn't on this machine, so these error numbers show the pipeline works, they're not results. Blue says nowcast, not forecast: the model uses sessions started in hour t to predict hour t. That's legitimate here because the simulator generates those sessions and genuinely knows them. It is not legitimate as a published forecasting result."

This is the strongest move available. Nobody in the room gets to catch you with it later, and everything after it is trusted.

2 The map, then zoom in

45 sec

"Real IGN basemap - the French national mapping agency, no API key. Six sites across Ile-de-France with 71 charge points. Cars drive in, plug in, charge, and drive away. Orange is arriving, green is charging, blue is leaving."

Pan and zoom - moving it makes obvious that it is a real map. Then **zoom to street level on a moving vehicle**. This is the detail worth showing: the cars follow the actual road network, not straight lines between points.

"Routing is shortest-path over the real OpenStreetMap road graph for Paris - about 4,500 junctions - computed locally. No external routing service, so nothing to rate-limit or go down mid-demo."

Then the point about granularity, which pre-empts an obvious question:

"Prediction happens at the site level, not per charge point - because that's the granularity the model was trained at. Every non-ACN entity in the corpus is a whole site. Showing per-station predictions would be inventing precision the model doesn't have."

3 The race against persistence

60 sec

Point at the two big numbers. This is the actual argument.

"The headline isn't the model's error. It's the model's error against persistence - just predicting that this hour equals last hour."

Then explain why, because this is the most technically interesting thing you have:

"Session energy is allocated proportionally across the hours a session spans. So every interior hour of a session gets exactly the same kWh. Aggregated across a site, this hour equals last hour exactly whenever no session starts or ends. With 4 to 8 hour connections, most consecutive hour-pairs are mechanical plateaus. Persistence is therefore a brutal baseline, and the model's real job is predicting the transitions - the starts and the stops."

Typical live figure: model 35 to 42% better than persistence on MAE.

4 The chart - where it works and where it fails

45 sec

"Grey area is actual load, blue is the model, orange is persistence. Watch orange: it's the blue line shifted one hour right. On the flat stretches it's almost perfect. On the climbs it's a full hour behind - and that's exactly the gap the model is earning."

Then find a bad hour in the per-site table and say it out loud. Pick any row where ERR is red:

"Bercy here - 135.8 actual, 105.1 predicted, error 30.7. That's a cluster of cars plugging in at once. The error concentrates at transitions, which is exactly what the allocation structure predicts."

SHOW A FAILURE DELIBERATELY

It is more convincing than hiding one. It proves you understand the model rather than having only plotted it - and it makes the persistence argument concrete instead of theoretical.

5 Close on what makes it real

20 sec

"Drop the real corpus into harmonized_sessions/, re-run training, and the amber badge turns blue. No code changes. Station coordinates are synthetic - the national IRVE register has real lat/lon, rated power and operator for every charge point in France, and that's a drop-in swap too."

Questions you will get

"Is this actually your model, or a mock-up?"

Real. Live XGBoost - binary: logistic classifier x reg: tweedie regressor at power 1.3, 500 trees each, 50 pinned feature columns, scored through the same pipeline. `hurdle_predict` the training script uses. Only the training data is synthetic. You can prove it on the spot:

```
python3 -c "from sim.model_service import ModelService; m=ModelService();
            print(len(m.columns), 'features | ', m.meta['data_source'])"
```

"How do you know the live features match training?"

All 38 features are asserted equal to the offline builder to 1e-9. It earns its keep: it caught `expanding_mean` drifting, because that feature expands over all history while the live ring buffer holds only 336 hours. It is now carried as explicit state.

```
python3 test_parity.py
```

"Are the cars actually driving on roads?"

Yes. The Paris road network is fetched once from Overpass, contracted from 65,000 shape points down to 4,558 junctions, and cached in `artifacts/roads.json`. Routing runs locally with `networkx` at about 10ms per route - no external service. Rebuild with `python3 -m sim.roads`.

"Is this SUMO?"

No - it is a purpose-built agent simulation. SUMO models car-following physics at the second level; this model predicts hourly energy per site, so sub-hour vehicle dynamics land in the same bucket and would not move the numbers. What the model needs is when cars arrive and how much they draw, which the agent layer produces directly. SUMO would earn its place for battery-physics or congestion-feedback claims.

"Is the simulation testing on data the model trained on?"

No. The warm-up buffer sits entirely inside the held-out window - zero rows from training - and every hour the model predicts is generated after the corpus ends. **Volunteer the caveat though:** the simulated fleet is drawn from the same generative process as the training corpus, so it is an in-distribution test. No drift, no regime change.

That is why the margin over persistence looks so stable.

"Why only 6 sites when France has 150,000 charge points?"

Because the model was trained on site aggregates. Per-station prediction needs per-station training history - polling the Belib' real-time feed and waiting about 10 weeks. Worth starting now; not fakeable today.

"Why is WAPE sometimes over 100%?"

Overnight, actual load approaches zero and WAPE divides by it. Watch MAE and RMSE at night, WAPE during the day. Say this before someone points at it.

"Can you make it run faster or slower?"

SIM_HOUR_WALL_SECONDS in sim/engine.py. Lower is faster.

Do not claim

FOUR THINGS THAT ARE NOT TRUE YET

1. That these error numbers are research results - they are synthetic-trained.
2. That it forecasts. It nowcasts, and the reason is a real finding about the feature set.
3. That the station positions are real.
4. That the model beats persistence *on the real data*. Nobody has run that comparison, because no baseline exists in the notebooks yet. It is the first thing to fix.

If something breaks

Symptom	Cause / fix
"accumulating hours..."	Normal for the first ~6 simulated hours. Wait.
Blank map, rail works	IGN tiles blocked - check network. The simulation itself is fine.
Cars move in straight lines	Road cache missing. Run <code>python3 -m sim.roads</code>
Page frozen, no errors	Check <code>curl -s localhost:8008/health</code> for status and error
FileNotFoundError: artifacts	Run <code>python3 train_for_sim.py</code> first
Port already in use	kill <code>\$(lsof -ti:8008)</code>
Empty map, everything 0.0	You started at 03:00 sim time. Wait for morning, or restart.
Layout stacked / cramped	Browser window under 900px wide. Widen it.